Jared Howse Lab 5

N	P = 1	P = 2	P = 4	P = 8
1000	0.015623	0.018970	0.016087	0.036813
5000	0.446577	0.599339	0.200761	0.350438
10000	1.84623	1.567827	0.889131	1.169187
20000	7.083932	3.333160	3.499028	4.145795

Speedup calculations

N	1	2	4	9	Speedup 2	Speedup 4	speedup 8
1000	0.015623	0.01897	0.016087	0.036813	0.823563521	0.9711568	0.4243881
5000	0.446577	0.599339	0.200761	0.350438	0.745115869	2.2244211	1.2743395
10000	1.84623	1.567827	0.889131	1.169187	1.177572526	2.0764432	1.5790716
20000	7.083932	3.33316	3.499028	4.145795	2.125290115	2.0245428	1.7087029

The program does not achieve linear speedup at all stages

- The limits to this scalability are largely the factors of communication overhead, bandwidth synchronization costs and load balancing. If our problem size is small adding more processes increases runtime because the resources needed to have the extra unnecessary processes communicate is a waste. If the problem size is matching the number of processes in an appropriate way we get linear speedup. This is a pattern for the last 3 problem sizes. If we hit the sweet spot we can even achieve superlinear speedup in some cases.

Sometimes increasing N can improve speedup because we are getting our moneys worth for using MPI meaning the problem size can make use of MPI in such a way that the communication and synch costs are worth it

You get crashes from extremely large N because the amount of memory you would need greatly exceeds what is available in your system and even more so available in your development environment so you try and write and access memory that just does not exist